

Design Token Architecture: Building a Scalable Style Foundation

Opening

Hello, everyone! You're here, hopefully, because you're looking to learn a bit more about scaling your styles - and you know that it's more than just your CSS.

Let me guess - Your CSS started simple, right?

You had an app...and everything looked good

Then you grew. You needed a second app.

You copied over your styles from the first app - everything still looks good.

Then came a third app. Something you made in your spare time - a POC

But everyone loved it...so you shipped it.

You made a bunch of decisions that made sense at the time.

Now that it's in production...some things just don't...add up.

Things are just a little different

It's fine, though. It doesn't get a lot of traffic.

And everything is fine. For a few years.

Now, suddenly, you have 5 apps

Your brand has evolved

Your teams have grown

And now...nothing is quite right

You have a primary blue...in three different forms...across 5 apps

Your spacing is..."consistent"

...it just differs here and there

And...at some point you open one app and think...

"Wait...are those **serif** fonts?"

The problem isn't your CSS

The problem...is that you don't have a shared language

You say "This should be an H1"...

But your team imagines something a little different than another team.

Your PM says "Let's tweak the hover state on this button"...

But...to what?

Your PM thinks it should go lighter

You made it darker

And neither of you is wrong...

You just aren't speaking the same language

And...when your team doesn't share a language...consistency isn't just hard – it's **impossible**.

The Scaling Problem

Having the shared language is easy when your a small team
Or...when you only have 1 app
But you've grown. Hey...did any of you think to write down what the primary font in your app was?
"I didn't" is the usual response
Or maybe "Bill knows all of that"...
Until Bill leaves and no one thought to ask
It wasn't part of the handoff
Stacy does what she remembers...
Ted's team has always done it their way...
And everything is...just...lost
Same UI...Just slightly different everywhere
When everyone does it the way they've always done it...
you have no shared source of truth
When everyone does it they way their code example showed it...
Then pretty soon, nothing means the same thing...to anyone
No one trusts the system...
So everyone...builds their own

So...
How do you scale, then, when styles have no meaning?
How do you give meaning to your styles so that you can scale?

Enter...Design tokens!

What are design tokens?

So what are design tokens?
Have you used them?
Do you use them?
Design tokens are more than variables
Design tokens are the decisions you have made, to share your intent
Design tokens are the what, and the why...and the how...of your UI
When used appropriately, they bring meaning to your code...to your design
When used incorrectly...they can cause confusion.
They can ****take your meaning away****
Design tokens are your foundation
But...How? How do they build your foundation?

To best achieve this, we break tokens into 3 groups:

Primitives
Semantics
And Component tokens

The Three-Layer Token Architecture

Primitives

Primitive tokens are our base level
We've all taken a physics class
Well...these are our atoms
By themselves...our primitive tokens have value
By themselves...they have **some** meaning
But...what we **build** with these atoms
That's where the **real value** comes from
These are easy
These are simple declarations of **what is**
That's it.
They're useful on their own...
But their real power shows up when you move from
"What is"
To "what it means"

So...how do we get from "this is what it is"
To "this is what it means"

Semantic Tokens

Semantic tokens is where things start to get interesting
If primitives are our atoms...Semantic tokens...are our molecules
This is where we move from "what is"...
To "what it means"
A primitive token says "This is blue"
A semantic token says "This is a background color"
It's the same value...but with a very different meaning.
Semantic tokens map your intent...to your value
Instead of "What color should i use"
You're asking "What is this for?"
Is this a background?
Is this text?
Is this a border?

And that's a much easier question to answer
Now your code is communicating intent
Not just a value
And this is where things get powerful
Because when your intent is clear
You can change the value...without changing the meaning
If your brand changes
You don't update your components
You update your semantics
This is the layer that gives your system flexibility
But...we're still not quite there yet
Because semantics define meaning
But they don't define usage
For that, we need one more layer

Component tokens

So Component tokens...
If primitives are atoms...
And semantics are molecules...
Component tokens...are where those molecules become real
This is where everything comes together
If primitives define what is...
And semantic tokens define what it means...
Component tokens define how it's used
Component tokens are specific
intentional
And as you might have guessed...
Scoped to a component

Instead of saying that this is the primary background color
You say this is the button background

Now, we're not just talking about meaning
Now we're talking about "actual usage"
This token doesn't just say "this is the background"

Because your components are no longer reaching into the system
They're being *fed* by it
Your components don't need to know what "blue" is
Your components don't need to know what the "primary background" is
It just needs to know
"What's *my* background?"
And that answer comes from a component token

And that token maps...
To a semantic token...
Which maps to a primitive
Primitives...
Semantics...
Components
Each layer has a single responsibility
And that's what makes the system scalable
Now, here's the best part
When you need to change something
You don't go hunting through your codebase
You change it...in only one place
Your triangle...is now complete
Each piece has a role.
Remove one...the system breaks
Let me show you what that actually looks like

** DEMO **

End Demo

What Belongs in a Design System

Documentation

What colors mean what?
How things *should* look
What does our spacing look like?
What about typography?
Who do you talk to for changes? Decisions?

Components

Only the *shared*, reusable ones belong
Anything too specific...or too temporary...does not belong

"No"

You can't include everything

'The system is *not open* to every change anyone wants

Saying "No"...that's how you maintain clarity

And...how you keep it scalability

More docs

Explain *why* some choices were made

Explain *why* some things were left out

This...is the secret to a good system

This is how you build trust

A design system is not just code...

It's a set of agreements...

A place for intentional decisions...

A design system is your shared language

The risk of doing too much/composition

Now that we know what belongs...Let's talk about what happens when you try to include everything

This is where design systems...fail

You can't capture every component...every style...every preference

Trying to do so creates clutter

It creates confusion

It creates a system nobody wants to use

This is why "No" is a feature...not a bug(pause

Composition is the hallmark of a good system

A design system is not about quantity...it's about *quality*

It's about how pieces fit together...intentionally

It's about composition

If your components compose well...everything scales

If they don't...your tokens, your components, your intent...all of it drifts

So discipline...and composition...are what keep a design system useful

And that's exactly what our token layers are designed to support

CSS Custom Properties as the Engine

So...why do we break it down like that?

Into variables?

Well...it lets us simplify our work

Overrides

It gives us easy overrides

Dark mode? Guess what...change your semantic

They're cascade-aware

When you change a primitive

Everything else updates

If you only want to change a component...

You swap a semantic token

Your change happens exactly where you want it

Without affecting everything else

Simplicity

Build steps? Nope.

No extra build step

No extra overhead

This is just...CSS

Theming

Add some structure...you get themes

Now, we need to talk about something serious for a minute

Once you start using these variables

Once you move to a token system

You have to use them everywhere

Because if you don't
You will start to introduce drift
Again
And we've already seen what that looks like

Practice doesn't make perfect
Perfect practice...makes perfect

Migration Strategy

CSS variables turn our tokens
From static values
Into a dynamic system
But...
How do we actually get there?

Step 1: Introduce tokens without deleting anything

This can be done slowly
This...should...be done intentionally
Pick one thing
Maybe that's your colors
Create your color tokens
The...start replacing your hardcoded values
There is no need to rush this

Step 2: Map to semantic tokens

Now you're introducing meaning to your system
Now...you're introducing...**intent**
And that intent needs to be...crystal...clear'
This is where your UI...starts...to document itself

Step 2.5 - Component tokens

Once your semantics are in place...

You do the same thing for your component tokens
Work one component at a time
Maybe this is a button...
Or a card
Map it to a component token that depends on your semantic tokens
Replace it's hard-coded styles with the new token
Test it in your app
The...rinse and repeat
One component at a time
Incremental
Intentional
Predictable
And slowly, your system grows

Step 3: Replace usage gradually

This isn't a race
This isn't "just some task" that needs completed
This is you bringing order...to chaos
This is how you make things manageable
This is how you make things easier
This is how you help your team...
And every team around you...
Move faster
Slow is smooth...
Smooth is fast
And if you're managing multiple apps,
It's faster...and easier to enforce consistency...
If you bundle all of your tokens as an npm package
Now your tokens aren't copy pasted...
They're **imported**
Every app imports the **same** source of truth
So now...everyone is using...
The same colors
The same spacing
The same typography
But a warning to everyone here:
Keep it clean and simple
Only export what's actually needed
Don't make it complicated.
You want it to be more complicated...**not** to use...this system

Which brings us to

Step 4: Code reviews...and linting

This is where the system becomes real
This only works
When everyone is on the same...page
This only helps...if everyone participates

This isn't something that you "have to do"
This *is* the system
It's no different than your other linting rules
It's no different than your branching strategy
Everyone follows it
So that everyone benefits from it

I want to take a second here and give you guys the key warning:
If you try to "big bang" this...you will fail
You will break things...
You will frustrate your team...
And people will stop trusting the system...again
This must be...*intentional*

Ok

Storybook

If tokens define the language
Then our components enforce that language

But...
How do we know that things are working?
We can use tools like Storybook

Our tokens...
Our very intentionally named tokens...
Are already doing a lot of documentation for our UI
But...Storybook lets you check that documentation
It lets us create new, living documentation that can be experienced

Stories become the way we check our contracts
You made a change?
Stories allow us to test that surface quickly

And...they're a great way to get feedback from stakeholders early...
Before you run your build process

Closing

Design tokens aren't about variables
Design tokens aren't a new framework
Design tokens are about giving your team a shared language
Design tokens are about defending your values ...your values
your meaning
and your usage
If your system separates those concerns...scaling becomes simple
If your system doesn't separate concerns...
If your system can't scale easily...
It drifts
But when you do it right...Your UI becomes clear...
Consistent...
And easy to maintain
And the next step is to let your components document themselves.
That's where tools like Storybook come in, which I'll cover in another session if you want to learn more

Q&A slide